



Advanced NLP

Session 4 - Autoregressive Language Modeling & Pre-training Objectives

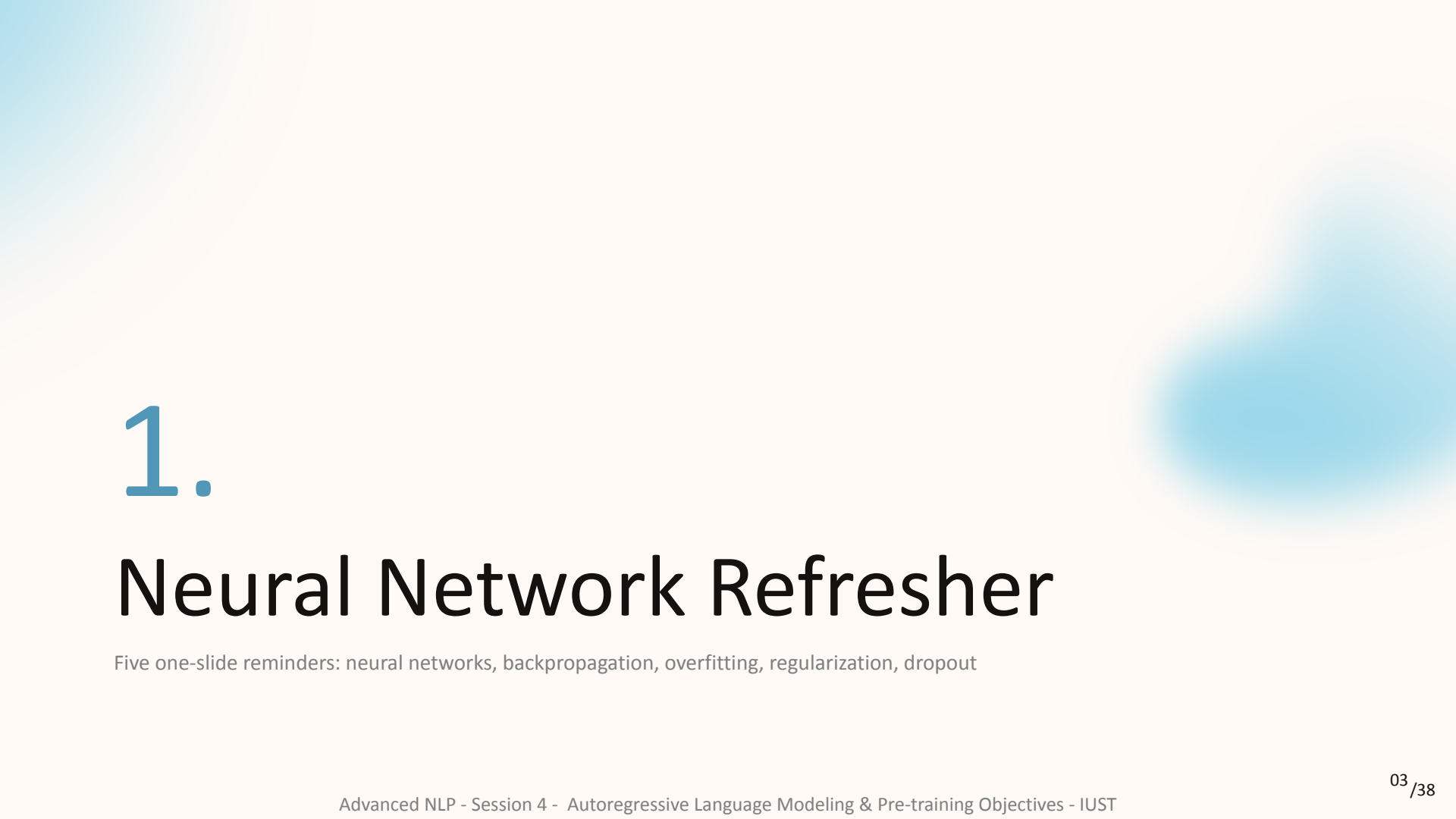
Teacher: Dr. Behrouz Minaei-Bidgoli

Presenter: Alireza Nazari

Presentation creator: Alireza Nazari

Contents

1. Neural Network Refresher
2. Optimizers
3. What is a Language Model?
4. n-gram Language Models
5. Neural Language Models



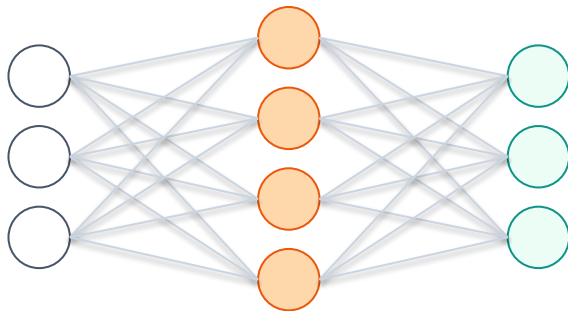
1.

Neural Network Refresher

Five one-slide reminders: neural networks, backpropagation, overfitting, regularization, dropout

Overview of neural networks

A 2-LAYER FEEDFORWARD NETWORK



input $x \in \mathbb{R}^d$ hidden $h = f(W_1 x + b_1)$ $\hat{y} = \text{softmax}(W_2 h + b_2)$

One neuron: $z = w^T x + b$, $a = f(z)$. A layer applies this to many neurons at once (matrix product). Depth = composition of layers. **Universal approximation:** one wide hidden layer can approximate any continuous function, but depth learns useful features far more efficiently.

Activation functions (the non-linearity f)

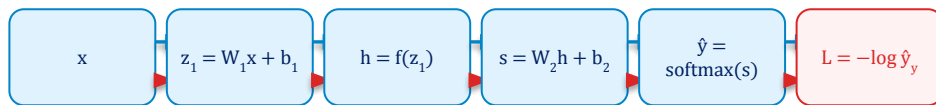
name	$f(z)$	derivative $f'(z)$
sigmoid σ	$1 / (1 + e^{-z}) \in (0,1)$	$\sigma(z)(1 - \sigma(z)) \leq 0.25$
tanh	$(e^z - e^{-z}) / (e^z + e^{-z}) \in (-1,1)$	$1 - \tanh^2(z) \leq 1$
ReLU	$\max(0, z)$	0 or 1

Why the non-linearity matters

- Without f , two linear layers collapse into one: $W_2(W_1 x) = (W_2 W_1)x$. No extra expressive power.
- Saturating functions (sigmoid, tanh) have **tiny derivatives** for large $|z|$: the source of vanishing gradients in deep and recurrent nets.

Backpropagation: computing all gradients in one backward pass

COMPUTATIONAL GRAPH: FORWARD (BLUE) AND BACKWARD (RED)



each node passes back: (upstream gradient) $\times$ (its own local gradient)

Forward pass: compute and cache every intermediate value. **Backward pass:** walk the graph in reverse applying the chain rule; every parameter gets its gradient in **one pass**, at a cost comparable to the forward pass.

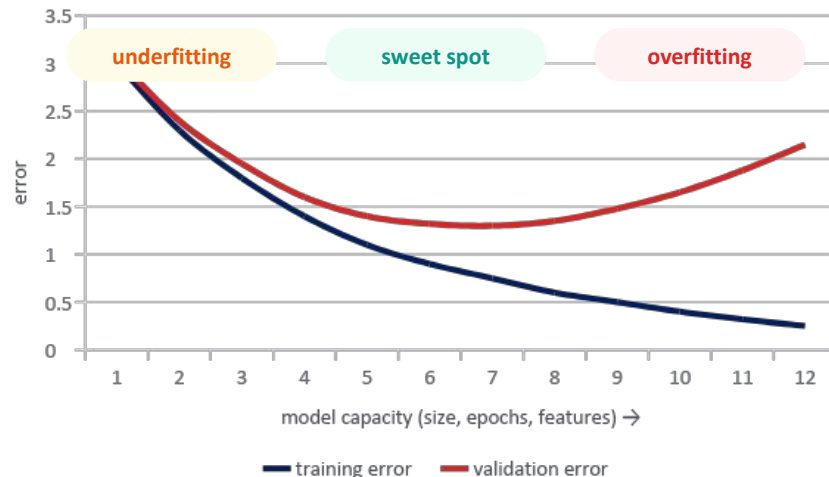
Chain rule, on a graph

$$\partial L / \partial W_1 = (\partial L / \partial s) (\partial s / \partial h) (\partial h / \partial z_1) (\partial z_1 / \partial W_1)$$

Local gradients are cheap: for $z = Wx + b$, $\partial z / \partial W = x^T$ and $\partial z / \partial x = W^T$; for $h = f(z)$, $\partial h / \partial z = f'(z)$ elementwise. Backprop **reuses** $\partial L / \partial s$ for every parameter upstream of s .

Overfitting: memorizing the sample instead of learning the distribution

VALIDATION ERROR vs MODEL CAPACITY (SCHEMATIC)



High bias (left): too simple, both errors high. **High variance** (right): fits noise, training error $\rightarrow 0$ while validation error rises.

Definition & symptoms

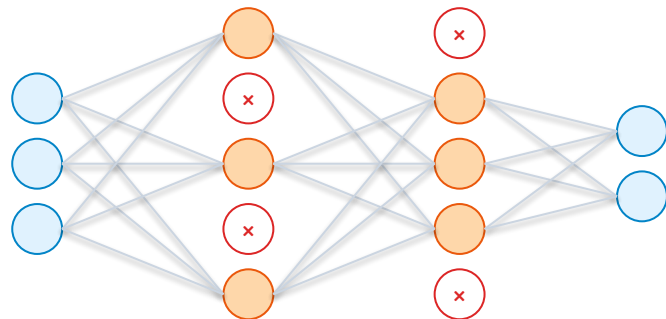
- **Symptom:** training loss $\downarrow$ while validation loss $\uparrow$ (or a large train/val gap).
- Risk grows with **more parameters, longer training, less data, noisier labels**.

What to do (details next slides)

- More / better data, data augmentation
- **Regularization:** weight decay, dropout, early stopping
- Reduce capacity or share parameters (as CNNs, RNNs do)

Dropout: train a random sub-network at every step

TRAINING: EACH STEP USES A RANDOM THINNED NETWORK



x = unit dropped for this step ($p = 0.4$ here); its outgoing weights get no gradient

The mechanism (inverted dropout)

Backprop passes through the same mask: dropped units send and receive **zero** gradient this step.

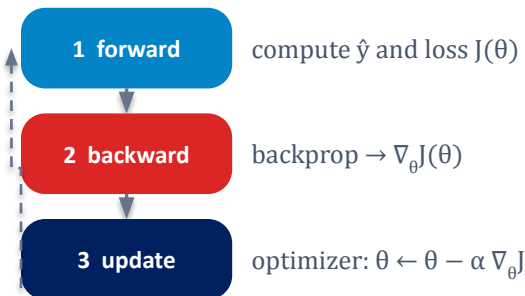
2.

Optimizers

From gradient descent to Adam and AdamW: how the weights actually get updated

What does an optimizer do?

THE TRAINING LOOP



repeat for many steps (an epoch = one pass over the training set)

Gradient descent, the basic update

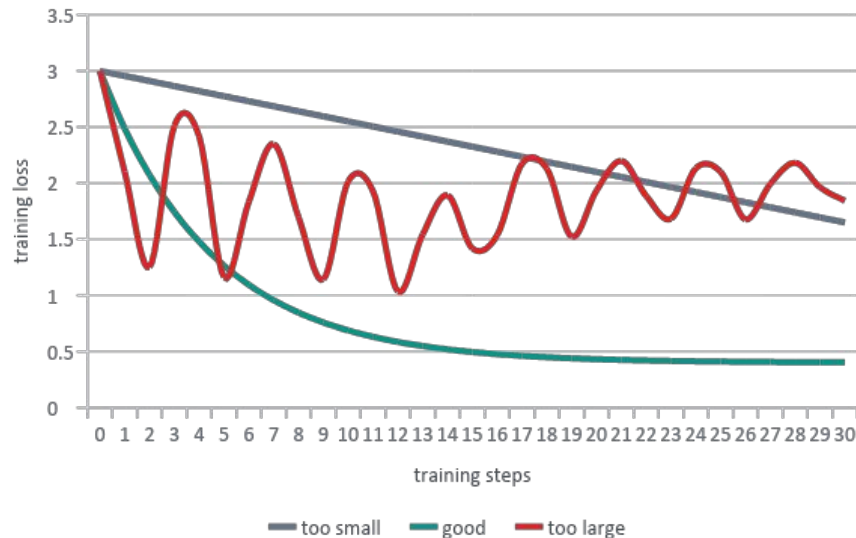
$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} J(\theta_t)$$

α = **learning rate** (step size). The gradient is the direction of steepest ascent, so we move against it. The optimizer's job: turn the gradient into a **good step**: right size, right direction, per parameter, despite noise.

variant	gradient computed on	cost per step / noise	in practice
Batch gradient descent	the whole training set (N examples)	very expensive; exact gradient	only for tiny datasets
Stochastic GD (SGD)	one example at a time	cheap; very noisy	poor GPU utilisation
Mini-batch SGD	B examples (B = 32 ... 4096)	Bx cheaper than batch; noise	the default : parallel hardware, smooths noise

The learning rate: the single most important hyperparameter

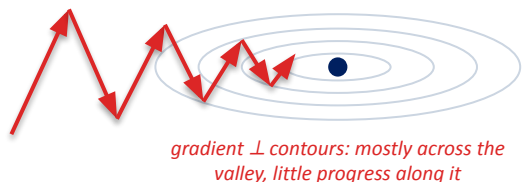
LOSS vs STEPS FOR THREE LEARNING RATES (SCHEMATIC)



Too small: slow, may stall on plateaus. **Good:** fast descent, then settles. **Too large:** oscillates around the valley, can diverge to Inf/NaN.

Why plain SGD struggles

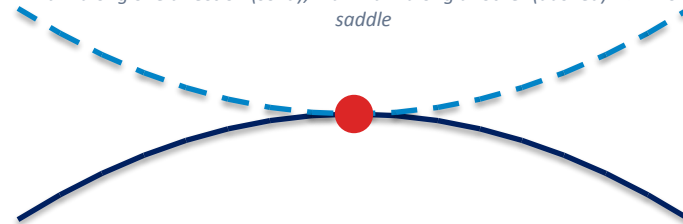
ILL-CONDITIONED "RAVINE": SGD ZIG-ZAGS



gradient $\perp$ contours: mostly across the valley, little progress along it

SADDLE POINTS & PLATEAUS: GRADIENT ≈ 0

minimum along one direction (solid), maximum along another (dashed): $\nabla J = 0$ at the saddle



In high dimensions saddle points vastly outnumber local minima; SGD with tiny gradients crawls across them. Momentum and adaptive steps help escape.

Ill-conditioning

Curvature differs by orders of magnitude across directions $\rightarrow$ one α is too big for steep directions and too small for flat ones.

$\rightarrow$ momentum, adaptive rates

Noisy gradients

Each mini-batch gives a noisy estimate; the trajectory jitters and the final loss oscillates instead of converging.

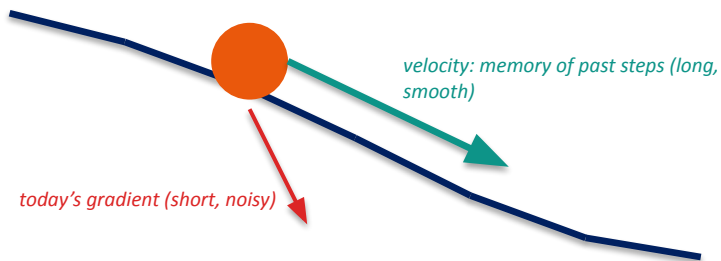
$\rightarrow$ momentum (averaging), LR decay

Momentum: let the ball keep rolling

DEFINITION

Momentum = a “velocity” that remembers past gradients. Each step moves along the velocity, not just along today’s gradient. Think of a **heavy ball rolling downhill**: it keeps going in the direction it has been going, and small bumps do not stop it.

THE HEAVY BALL



How it works, in words

- 1 **Remember:** new velocity = mostly the old velocity (90%) + today's gradient
- 2 **Move:** step along the velocity, not along today's gradient alone
- 3 **Result:** agreeing directions add up ; flip-flopping directions cancel

SAME VALLEY: PLAIN SGD (RED) vs MOMENTUM (GREEN)

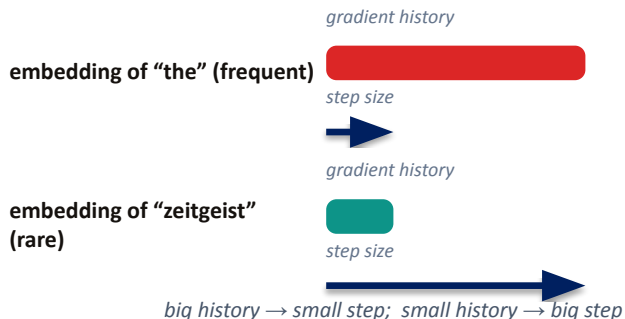


AdaGrad: a personal learning rate for every parameter

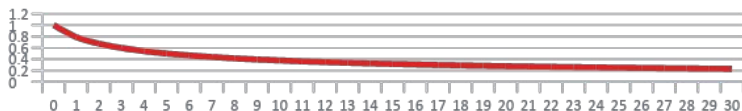
DEFINITION

AdaGrad gives **each parameter its own learning rate**: the more gradient a parameter has received in the past, the **smaller** its steps become. Frequently-updated parameters slow down; **rarely-updated parameters keep taking big steps**.

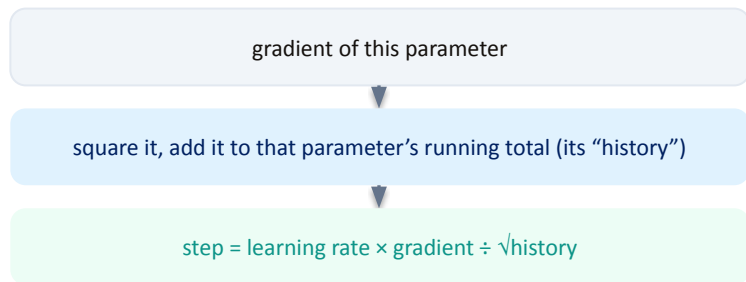
TWO PARAMETERS, TWO DIFFERENT STEP SIZES



effective step size over time



What happens at every step



THE FLAW

The history **only grows**, so every step size shrinks toward zero: **learning stops early**, even far from the minimum. Fix: let the history forget → **RMSProp**.

RMSProp: the same idea, with a short memory

DEFINITION

RMSProp = AdaGrad with a **short memory**. The step size of each parameter is set by its **recent** gradients (roughly the last 10 steps), not by everything since the start. Old gradients **fade out**, so the step size adapts to where we are now.

HOW MUCH EACH PAST GRADIENT COUNTS

AdaGrad



every past gradient counts fully, forever

RMSProp

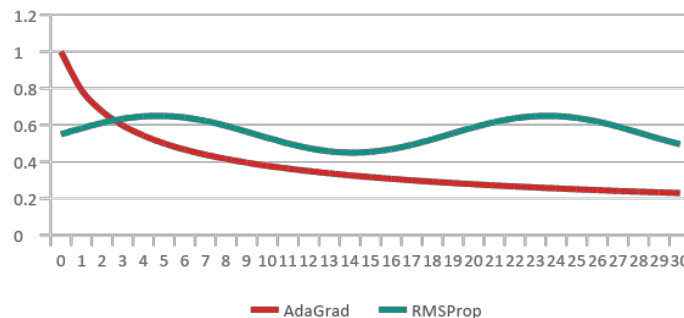


old gradients fade, recent ones count most

← oldest

newest →

effective step size over time



Kept from AdaGrad

each parameter still has its own step size: rare parameters take bigger steps

What changed

“total since the start” → “moving average of recent gradients”: the step size no longer dies out

Still missing

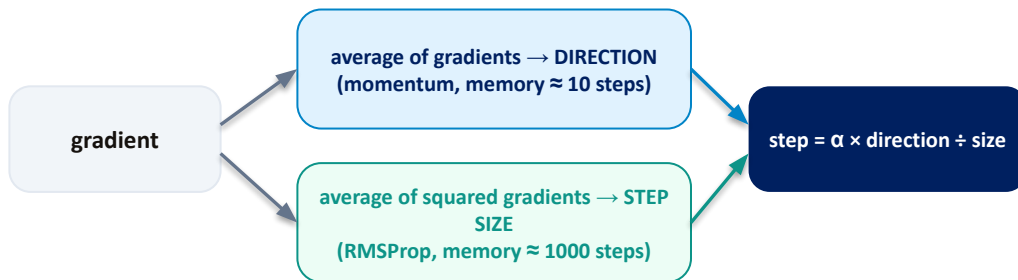
no memory of **direction** (no momentum): the path still jitters → Adam adds it

Adam: momentum + RMSProp + a warm-start fix

DEFINITION

Adam = **momentum** (which way to go, from the average of recent gradients) + **RMSProp** (how big a step, per parameter, from the average of recent squared gradients) + a **warm-start fix** so the first steps are not too small.

WHAT ADAM COMPUTES AT EVERY STEP



direction is smoothed (no zig-zag); size is normalized per parameter (rare parameters still move)

The warm-start fix (bias correction)

both averages start at 0, so the first estimates are far too small

first step, raw



first step, corrected



Adam scales the early estimates up, so step 1 already has the right size

3.

What is a Language Model?

Main Resources: Stanford CS224N Lecture 4 (Diyi Yang) • CMU CS11-711 ANLP Lecture 3 (Sean Welleck) • CS224N Lecture Notes Part V

Three types of prediction: binary, multi-class, structured

01**Binary classification***Two classes*

I hate this movie

positive

negative

02**Multi-class classification***A small, fixed set of classes*

I hate this movie

very good / good / neutral

bad / very bad

03**Structured prediction***Exponential / infinite label space*

I hate this movie

PRP VBP DT NN

この映画が嫌い

Where does language modeling fit?

A language model outputs a **whole sequence** of tokens. Its “label space” is every possible sentence, which is practically infinite. Language modeling is therefore **structured prediction**.

Language Modeling: predicting what word comes next

EXAMPLE

the

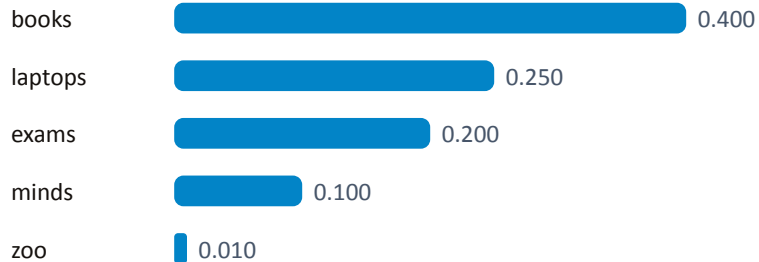
students

opened

their

—

Probability of the next word (illustrative)



Every word in the vocabulary V gets some probability, even “zoo”. Predicting the next word means producing this entire distribution, not a single answer.

FORMAL DEFINITION

Given a sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$, compute the probability distribution of the next word $x^{(t+1)}$:

$$P(x^{(t+1)} \mid x^{(t)}, \dots, x^{(1)})$$

- $x^{(t+1)}$ can be **any word in the vocabulary** $V = \{w_1, \dots, w_{|V|}\}$
- A system that does this is called a **Language Model (LM)**
- This is “the most important concept in the class”: it leads to most of modern NLP
- The same object can be read as a **classifier with $|V|$ classes** at every position

A Language Model assigns a probability to any text

CHAIN RULE: FROM NEXT-WORD PROBABILITIES TO TEXT PROBABILITY

$$P(x^{(1)}, \dots, x^{(T)}) = P(x^{(1)}) \times P(x^{(2)} | x^{(1)}) \times \dots \times P(x^{(T)} | x^{(T-1)}, \dots, x^{(1)})$$

$$= \prod_{t=1}^T P(x^{(t)} | x^{(t-1)}, \dots, x^{(1)})$$

next token

context

A distribution over all sequences

Biased coin: $P(X) = 0.6$ if $x = 1$, 0.4 if $x = 0$. Two outcomes.

Language model: $P(X)$ over every possible token sequence. One square = one sequence; there are astronomically many.

$P(X) = 0.000013$ if x is "a.",
 0.000001 if x is "aa.", ...,
 0.019100 if x is "a cat sat."

SCORING SEQUENCES

$P(\text{Jane went to the store .})$

high

$P(\text{store to Jane went the .})$

low

Grammatical, meaningful text should receive high probability; scrambled text should receive low probability. This is exactly what speech recognizers and translation systems use to pick among candidate outputs, e.g. preferring "the cat is small" over "small the is cat".

You use Language Models every day. Why should we care?

EVERYDAY LANGUAGE MODELS

I'll meet you at the

office

airport

usual

what is the

what is the weather today

what is the capital of iran

what is the time in tokyo

what is the meaning of life

Search autocomplete completes your query

Predictive typing: the LM proposes the next word

- Speech recognition
- Spelling & grammar correction
- Machine translation

Why should we care about Language Modeling?

- **Benchmark task:** it helps us measure our progress on predicting language use.
- **Sub-component** of many NLP tasks, especially those that generate text or estimate the probability of text: predictive typing, speech recognition, handwriting recognition, spelling/grammar correction, authorship identification, machine translation, summarization, dialogue, ...

THE BIG PICTURE

*"Everything else in NLP has been rebuilt upon Language Modeling: **ChatGPT is an LM!**"*

— Stanford CS224N, Lecture 4

4.

n-gram Language Models

The pre-deep-learning answer: count how often word sequences occur in a large corpus

n-gram Language Models: learn from counts

QUESTION → ANSWER

How to learn a Language Model? Answer (pre-deep learning): learn an **n-gram Language Model**. An **n-gram** is a chunk of n consecutive words.

the

students

opened

their

—

unigrams $n = 1$

"the"

"students"

"opened"

"their"

bigrams $n = 2$

"the students"

"students opened"

"opened their"

trigrams $n = 3$

"the students opened"

"students opened their"

four-grams $n = 4$

"the students opened their"

Idea: collect statistics about how frequent different n-grams are in a large corpus and use these to predict the next word.

The Markov assumption and counting

1 Markov assumption

$x^{(t+1)}$ depends only on the preceding $n-1$ words

$$P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)}) \approx P(x^{(t+1)} | x^{(t)}, \dots, x^{(t-n+2)})$$

2 Conditional probability

probability of the n -gram over probability of the $(n-1)$ -gram

$$= P(x^{(t+1)}, x^{(t)}, \dots, x^{(t-n+2)}) / P(x^{(t)}, \dots, x^{(t-n+2)})$$

3 Statistical approximation

count them in some large corpus of text

$$\approx \text{count}(x^{(t+1)}, x^{(t)}, \dots, x^{(t-n+2)}) / \text{count}(x^{(t)}, \dots, x^{(t-n+2)})$$

Special case: bigram ($n = 2$)

one-token context

$$P(X) \approx \prod_{t=1}^T p_{\theta}(x_t | x_{t-1})$$

$$p(x_t | x_{t-1}) = \text{count}(x_{t-1}, x_t) / \sum_{x'} \text{count}(x_{t-1}, x')$$

We can view this as **training the parameters**

$\theta_{ij} = p(x_j | x_i)$: one number per (previous token, next token) pair.

HOW DO WE GET THESE PROBABILITIES?

By counting them in some large corpus of text! The n -gram model is fully specified by a table of counts. Training is counting; prediction is a table lookup and a division.

Example: a 4-gram Language Model

Suppose we are learning a 4-gram Language Model: only the last $n - 1 = 3$ words matter.



$$P(w \mid \text{students opened their}) = \text{count}(\text{students opened their } w) / \text{count}(\text{students opened their})$$

SUPPOSE THAT IN THE CORPUS

"students opened their" occurred 1000 times

"students opened their books" occurred 400 times

→ $P(\text{books} \mid \text{students opened their}) = 0.4$

"students opened their exams" occurred 100 times

→ $P(\text{exams} \mid \text{students opened their}) = 0.1$

DISCUSSION

Should we have discarded the "proctor" context?

The clock, the proctor... a human reader knows the answer is "exams". A 4-gram model cannot see that far back. Longer context is needed, but as we will see, increasing n creates its own problems.

Bigram models in code: character-level names

COUNT MATRIX $\text{count}(x_{t-1}, x_t)$ (illustrative values)

	[S]	a	b	e	j	n
[S]	0	42	11	20	15	30
a	92	3	15	8	1	60
b	5	20	0	10	0	2
e	70	9	5	4	0	25
j	0	40	0	10	0	0
n	88	30	0	20	0	12

rows: previous character x_{t-1} • columns: next character x_t

Normalize each row $\rightarrow \theta_{ij} = p(x_i | x_j)$. Dark cells are frequent transitions (e.g. “n” is often followed by the end symbol; names often end in “n”).

Problem setup

- **Goal:** model a data distribution, i.e. $p_\theta \approx p_{\text{data}}$
- We only have a **dataset of samples** from p_{data} : $D = \{x_n\}$, $n = 1 \dots N$
- Split the dataset into **training, dev (validation), and test** sets
- Character-level tokenization: each name is a sequence of letters between start and end symbols

Code: github.com/cmu-l3/anlp-spring2026-code $\rightarrow$ 03_lm_fundamentals/lm_basics_bigrams.ipynb (based on Andrej Karpathy's makemore video)

Worked example: estimating a bigram model from a tiny corpus

TRAINING CORPUS (3 sentences)

<s> the cat sat </s>

<s> the cat ran </s>

<s> a dog sat </s>

Bigram counts (first → second):

count(<s>, the) = 2, count(<s>, a) = 1

count(the, cat) = 2

count(cat, sat) = 1, count(cat, ran) = 1

count(a, dog) = 1, count(dog, sat) = 1

count(sat, </s>) = 2, count(ran, </s>) = 1

Row totals: <s>: 3, the: 2, cat: 2, a: 1, dog: 1, sat: 2, ran: 1

ESTIMATED PROBABILITIES $p(v | u) = \text{count}(u, v) / \sum_{v'} \text{count}(u, v')$

$p(\text{the} \text{<s>}) = 2/3$	$p(a \text{<s>}) = 1/3$	$p(\text{cat} \text{the}) = 2/2 = 1$	$p(\text{dog} a) = 1$
$p(\text{sat} \text{cat}) = 1/2$	$p(\text{ran} \text{cat}) = 1/2$	$p(\text{sat} \text{dog}) = 1$	$p(\text{</s>} \text{sat}) = 2/2 = 1$
$p(\text{</s>} \text{ran}) = 1$	$p(\text{dog} \text{the}) = 0/2 = 0$	$p(\text{cat} a) = 0$	... all unseen pairs = 0

Each row of the table is a probability distribution over the next token (rows sum to 1). This is the trained model: $\theta_{u,v} = p(v | u)$.

Score a new sentence

$P(\text{<s> the cat ran </s>})$

$= p(\text{the} | \text{<s>}) \cdot p(\text{cat} | \text{the}) \cdot p(\text{ran} | \text{cat}) \cdot p(\text{</s>} | \text{ran})$

$= 2/3 \cdot 1 \cdot 1/2 \cdot 1 = 1/3$

Sparsity in one line

$P(\text{<s> the dog sat </s>})$

$= 2/3 \cdot p(\text{dog} | \text{the}) \cdot 1 \cdot 1 = 2/3 \cdot 0 = 0$

A fine sentence gets probability 0 (log = $-\infty$, infinite perplexity) because one bigram was never observed.

Two practical notes: log space and autoregressive generation

Note: using log space

multiplication of probabilities $\rightarrow$ addition of log probabilities

$$P(X) = \prod_{i=1}^{|X|} P(x_i)$$



$$\log P(X) = \sum_{i=1}^{|X|} \log P(x_i)$$

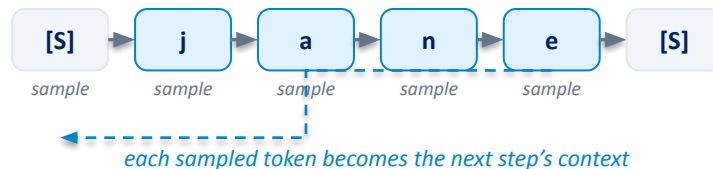
- **Numerical stability:** a product of many small probabilities underflows to 0; the sum of logs does not.
- Other conveniences: log-likelihoods add across tokens and examples; gradients of sums are simple.
- All losses in this course (NLL, cross-entropy) are written in log space.

Generation from an autoregressive model

iteratively sample a next token, then append it to the context

Until [S] (end symbol) is generated:

$$\hat{x}_t \sim p_\theta(x_t | \hat{x}_{t-1})$$



Equivalent to **sampling from the model's joint distribution** over full sequences: the chain rule works in the generative direction too.

More on decoding strategies (greedy, beam search, temperature, top-p) in Session 11.

Sparsity problems with n-gram Language Models

Sparsity Problem 1

Problem: What if “students opened their w” never occurred in the data? Then w has probability 0!

(Partial) solution: add a small δ to the count for every $w \in V$. This is called **smoothing**.

add-one (Laplace) smoothing, from CMU

$$p(x_t | x_{t-n+1:t-1}) = [1 + \text{count}(x_{t-n+1:t-1}, x_t)] / [|V| + \sum_{x'} \text{count}(x_{t-n+1:t-1}, x')]$$

Sparsity Problem 2

Problem: What if “students opened their” never occurred in the data? Then we can’t calculate the probability for any w!

(Partial) solution: just condition on “opened their” instead. This is called **backoff**.

backoff chain: shorter and shorter contexts

$$P(w | \text{students opened their}) \rightarrow P(w | \text{opened their}) \rightarrow P(w | \text{their})$$

Note: increasing n makes sparsity problems worse. Typically we can’t have n bigger than 5. Smoothing is a whole sub-field (Kneser-Ney, Good-Turing, ...), but none of it removes the fundamental limitation of counting exact word sequences.

Storage problems, and n-grams in practice

Storage problem

Need to store the **count for all n-grams** you saw in the corpus.

Increasing n or increasing the corpus **increases model size!**

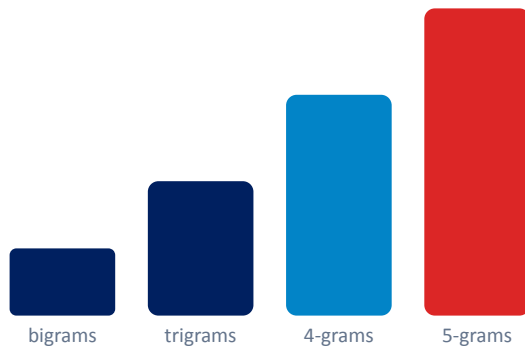


table size grows with n (schematic)

n-gram Language Models in practice

You can build a simple **trigram LM** over a **1.7-million-word corpus (Reuters, business & financial news)** in a few seconds on your laptop.



company		0.153
bank		0.153
price		0.077
italian		0.039
emirate		0.039
...		

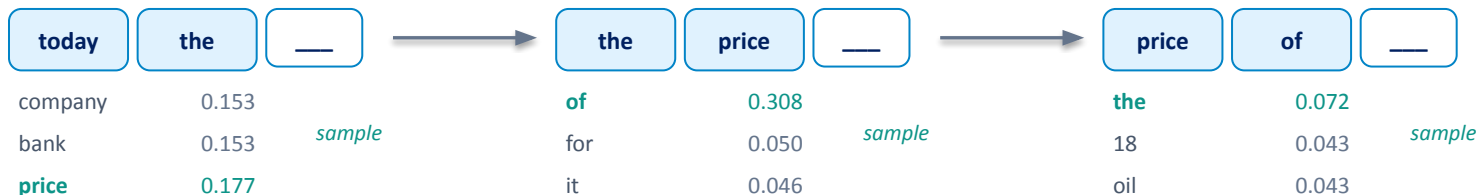
SPARSITY

not much granularity in the probability distribution

Otherwise, seems reasonable! Try for yourself: nlpforhackers.io/language-models/

Generating text with an n-gram Language Model

REPEATED SAMPLING: CONDITION → GET DISTRIBUTION → SAMPLE → SLIDE THE WINDOW



the sampled word is appended and becomes part of the next context

Generated with the Reuters trigram model:

"today the price of gold per ton , while production of shoe lasts and shoe industry , the bank intervened just after it considered and rejected an imf demand to rebuild depleted european stocks , sept 30 end primary 76 cts a share ."

Surprisingly grammatical!

...but **incoherent**. We need to consider more than three words at a time if we want to model language well.

But increasing n worsens the sparsity problem and increases model size...

What n-grams cannot do, and when to still use them

Cannot share strength among similar words

→ solution: neural networks

she bought a car

she bought a bicycle

she purchased a car

she purchased a bicycle

Cannot condition on context with intervening words

→ solution: neural networks

Dr. Jane Smith

Dr. Gertrude Smith

Cannot handle long-distance dependencies

→ solution: recurrent / attention models
(next sessions)

for tennis class he wanted to buy his own racquet

for programming class he wanted to buy his own computer

When to use n-gram models?

- Neural LMs achieve better performance, but n-grams are **extremely fast** to estimate and apply
- **Perfect memorization** can be useful

5.

Neural Language Models

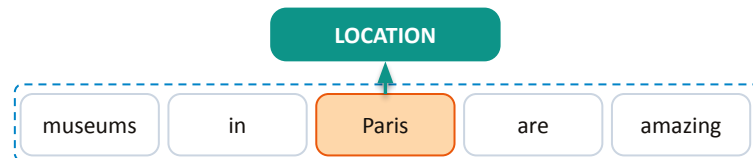
Replace the count table with a neural network: Bengio et al. (2003), A Neural Probabilistic Language Model

How to build a neural Language Model? Start with a window

Recall the Language Modeling task

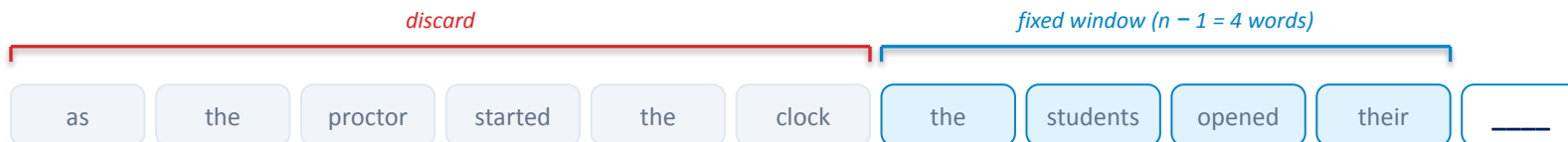
- **Input:** sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$
- **Output:** probability distribution of the next word $P(x^{(t+1)} \mid x^{(t)}, \dots, x^{(1)})$
- **How about a window-based neural model?** We saw this applied to Named Entity Recognition.

WINDOW-BASED NER (RECALL SESSION 3)



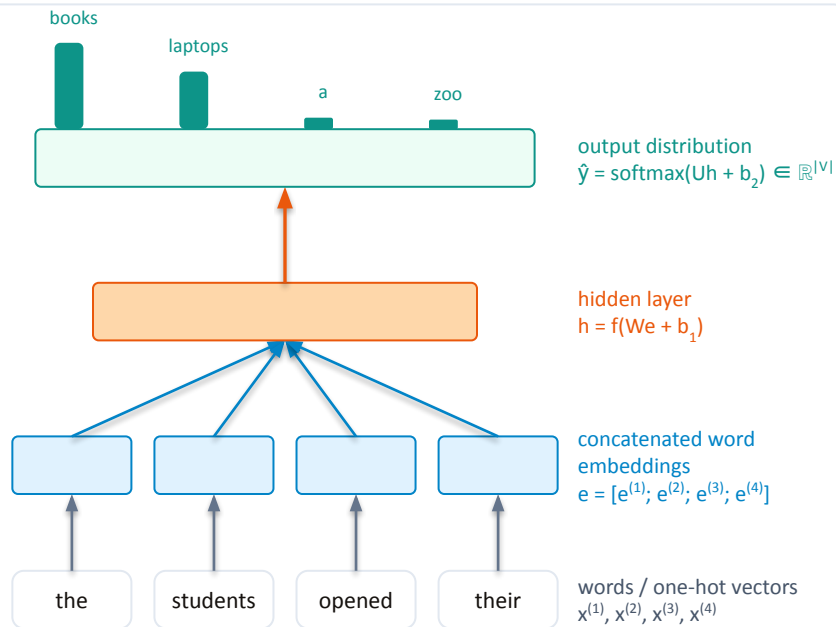
a window of words around the target → a classifier predicts the tag

A FIXED-WINDOW NEURAL LANGUAGE MODEL: THE SAME TRICK



Only the last few words enter the network. The window words are embedded, concatenated, passed through a hidden layer, and a softmax over the vocabulary produces the distribution of the next word. Approximately the model of **Y. Bengio et al. (2000/2003): A Neural Probabilistic Language Model**, the first large-scale deep learning model for NLP.

A fixed-window neural Language Model



Simplified view of Bengio et al. (CS224N Notes, Figure 2). The bars above the output layer are the predicted probabilities for candidate next words.

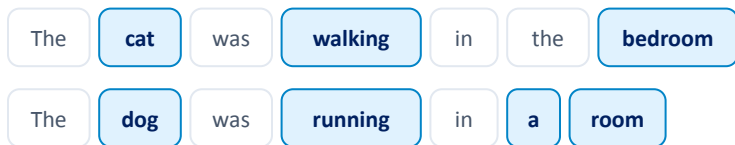
Original Bengio et al. (2003) equation

$$\hat{y} = \text{softmax}(W^{(2)} \tanh(W^{(1)}x + b^{(1)}) + W^{(3)}x + b^{(3)})$$

x = concatenated input word vectors. $W^{(1)}$ feeds the tanh hidden layer, $W^{(2)}$ maps hidden to output, and $W^{(3)}$ is a **direct linear path** from the word vectors to the output.

Why neural? Learned, distributed representations

n-gram LMs ignore similarity of words and contexts



Solution: use learned, distributed representations (Bengio et al., 2003)

Words become vectors; similar words end up close

cat	+0.7	-0.2	+0.5	+0.1
dog	+0.6	-0.3	+0.5	+0.2
bedroom	-0.4	+0.8	+0.1	-0.6
room	-0.3	+0.7	+0.2	-0.5

↑ shared knowledge transfers between neighbours

Improvements over n-gram LMs

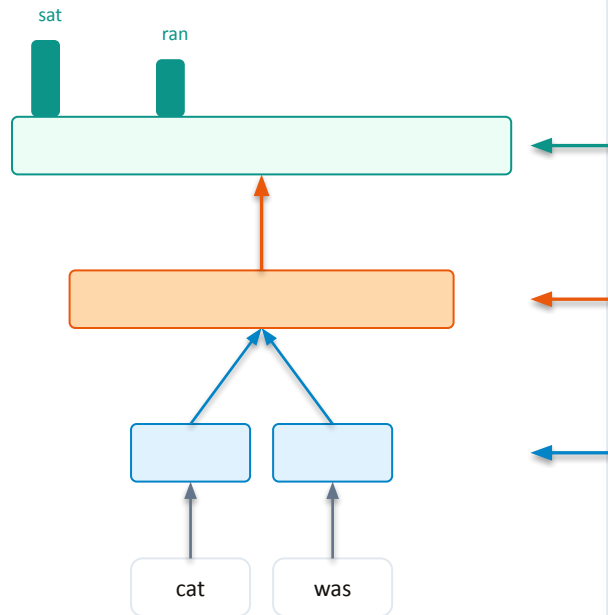
- **No sparsity problem:** every context maps to a distribution, even unseen ones
- **Don't need to store all observed n-grams:** the parameters are E, W, U and biases, not a table
- Similar words and similar contexts share statistical strength

Remaining problems

- **Fixed window is too small;** enlarging the window enlarges W
- The window can **never be large enough!**

We need a neural architecture that can process any length input
→ RNNs (next session)

Where is statistical strength shared?



Similar output words get similar output weights

The row w of the output matrix acts like a “template” for a word; words used in the same slots learn similar rows.

Similar contexts get similar hidden states

$$p_j^w = p(w | h_j) = (1/Z_j) \exp(w^T h_j)$$

$$p_k^w = p(w | h_k) = (1/Z_k) \exp(w^T h_k)$$

$$p_j^w / p_k^w = (Z_k/Z_j) \exp(w^T (h_j - h_k))$$

“It’s a great movie” vs “It is a wonderful movie”: the ratio is 1 when $w^T (h_j - h_k) = 0$, so training **makes hidden vectors $h_{\{j\}}$ and $h_{\{k\}}$ close to each other** when they predict the same words.

Similar words get similar embeddings, and rows of W_1 capture combination features

e.g. the 34th row of W_1 can output a positive number if the previous word is a determiner **and** the second-to-previous word is a verb (“giving a ...”).

Scorecard: which n-gram problems does the neural LM fix?



Cannot share strength among similar words

she bought a car / she purchased a bicycle

solved, and similar contexts as well!



Cannot condition on context with intervening words

Dr. Jane Smith / Dr. Gertrude Smith

solved!



Cannot handle long-distance dependencies

for tennis class he wanted to buy his own racquet

not solved yet: the window is fixed

RECAP SO FAR

Bigram LMs and fundamental concepts → n-gram LMs: count-based → neural network LM: learned representations. **Next:** practical training concepts (Section 6); the **any-length** architecture (RNNs) comes next session.

Advanced NLP - Session 4 - Autoregressive Language Modeling & Pre-training Objectives - IUST

NN Refresher

Optimizers

Language Modeling

n-gram Models

Neural LMs

Practical Training